Pipeline Walkthrough

Probabilistic Motor Subtype Progression Model for Parkinson's Disease

A step-by-step account of how every result was generated

Prepared as a study and defence guide

August 2026

Abstract

This document reverse-engineers the implementation in this repository and traces every reported number back to the code that produced it. Nothing here is generic methodology: each count was obtained by instrumenting the actual functions and recording the data shape before and after every transformation.

Where a design decision has a documented rationale, that rationale is quoted. Where it does not, this is stated explicitly rather than filled in with a plausible-sounding reason.

Contents

1 Current project state **2**
- 1.1 What exists . . . 2
- 1.2 What has actually been executed . . . 2
- 1.3 Which script produced which output . . . 2
- 1.4 Dead code and unused configuration . . . 2

2 Execution pipeline **3**
- 2.1 The order is proven, not assumed . . . 3
- 2.2 Module dependency graph . . . 3

3 Stage 0 — the raw inputs **3**
- 3.1 The four files the pipeline opens . . . 3
- 3.2 Why these sixteen items? . . . 5

4 Stage 1 — `01_build_panel.py` **5**
- 4.1 A. Purpose . . . 5
- 4.2 B. Inputs . . . 5
- 4.3 C. Code walkthrough with actual counts . . . 5
- 4.4 D. Data shape after each step . . . 7
- 4.5 E. Validation (`validate.py`) . . . 8
- 4.6 F. Research reasoning . . . 8

5 Target / label construction **9**
- 5.1 The formula, exactly as implemented . . . 9
- 5.2 Step 1 — cleaning (`clean_items`, `subtype.py`:39) . . . 9
- 5.3 Step 2 — subscores with a missingness policy (`_subscore`, line 62) . . . 9
- 5.4 Step 3 — classification (`classify`, line 121) . . . 9
- 5.5 Edge cases, and how each is resolved . . . 10
- 5.6 A fully worked example from the real data . . . 10
- 5.7 The ratio is not always defined . . . 11

6 Longitudinal panel construction **11**
6.1 The time axis (`_add_time_axis`, `panel.py`:97) . . . 11
6.2 What one patient's trajectory looks like . . . 11
6.3 Patients with too few usable visits . . . 12

7 Transition construction **12**
7.1 `consecutive_pairs()` (`transitions.py`:16) . . . 12
7.2 The same patient, transformed . . . 12
7.3 Arithmetic check on the pair count . . . 13
7.4 From counts to probabilities (`transition_matrix`, line 47) . . . 13

8 Statistical analysis **13**
8.1 Wilson score intervals (`transition_counts_with_ci`, line 67) . . . 13
8.2 Interval stratification (line 106) . . . 14
8.3 Reversion analysis (`reversion_after_change`, line 198) . . . 14
8.4 Kaplan–Meier (`first_entry` + `kaplan_meier`, lines 246, 299) . . . 14
8.5 Pathway analysis (`state_sequences` + `collapse_runs`, lines 177, 185) . . . 15

9 Trace of every major result **15**

10 Complete data funnel **17**

11 Code ↔ science map **18**

12 Professor defence questions **19**

13 What we have actually achieved **21**

14 What comes next **21**

1 Current project state

1.1 What exists

Twenty-four tracked files: seven modules in `src/ppmi_subtype/` (1,552 lines), three scripts, three test files, one configuration file, and documentation.

Location	Lines	Role
`src/ppmi_subtype/`	1,552	All analysis logic
`scripts/`	596	Orchestration: call `src`, print, save
`tests/`	598	74 tests, all exercising `src`

Table 1: Code inventory. The scripts contain no analysis logic.

1.2 What has actually been executed

Established by comparing file modification times, not assumed:

Item	Last modified
Newest source file (`scripts/00_data_audit.py`)	2026-08-21 11:15
All 20 output files	2026-08-21 11:17

Every output post-dates every source file, so **no output is stale**. All 74 tests pass against the same code that produced those outputs.

1.3 Which script produced which output

Output file	Produced by	Line
`outputs/visit_panel.csv`	`01_build_panel.py`	118
`outputs/baseline.csv`	`01_build_panel.py`	119
`outputs/panel_summary.txt`	`01_build_panel.py`	94
`outputs/validation.txt`	`01_build_panel.py`	112
`outputs/transitions/*.csv` (9)	`02_transition_analysis.py`	113–222
`outputs/transitions/report.txt`	`02_transition_analysis.py`	270
`outputs/transitions/figures/*.png` (6)	`plots.py` via stage 2	59

1.4 Dead code and unused configuration

Three findings, stated plainly because a reviewer will find them:

1. `config.yaml` line 19 defines `participant_status:`. A grep across `src/`, `scripts/` and `tests/` returns zero references. It is a configured path with no consumer.
2. Of the seventeen files under `data/`, exactly **four** are opened by the pipeline. The remainder — demographics, cohort history, five dictionary files, and four of the six files in `data/validation/` — are human reference material that no code path reads.
3. `scripts/00_data_audit.py` contains no `to_csv` or `write_text` call. It writes nothing. It is a read-only diagnostic, *not* a pipeline stage.

Not established from the current repository: there are no logs, no notebooks, no shell scripts and no run manifests. Execution history is inferable only from file timestamps and git commits.

2 Execution pipeline

2.1 The order is proven, not assumed

The numeric filename prefixes suggest an order, but the order is *established* by a data dependency: `scripts/02_transition_analysis.py` line 85 reads `outputs/visit_panel.csv`, and the only code that writes that file is `scripts/01_build_panel.py` line 118.

```
        +-----------------------------------------+
        |  00_data_audit.py    (DIAGNOSTIC ONLY)  |
        |  reads raw data, writes nothing         |
        +-----------------------------------------+
                (optional; off the critical path)

RAW DATA --> 01_build_panel.py --> outputs/visit_panel.csv
                                              |
                                              v
                                  02_transition_analysis.py
                                              |
                                              v
                                       outputs/transitions/
```

2.2 Module dependency graph

Derived from the `import` statements; the graph is acyclic.

```
config.py        <- foundation; imports nothing internal
    ^
    +-- loaders.py      (config)
    +-- subtype.py      (config)
    +-- validate.py     (config, loaders)
    +-- panel.py        (config, loaders, subtype)

transitions.py   <- imports NOTHING internal. Pure DataFrame functions.
plots.py         <- imports NOTHING internal. Pure DataFrame functions.
```

`transitions.py` and `plots.py` do not read the configuration at all. They take DataFrames in and return DataFrames or figures out. That is why they carry the heaviest test coverage: they have no hidden state.

3 Stage 0 — the raw inputs

3.1 The four files the pipeline opens

File 1 — the visit skeleton

`data/clinical/PPMI_Curated_Data_Cut_Public_20260511.xlsx`, sheet `20260511`.

Shape	**19,450 rows × 212 columns**
Unique `PATNO`	4,788
Distinct `EVENT_ID` values	18
Duplicate `(PATNO, EVENT_ID)`	**0**

One row = one patient at one scheduled study visit.

PATNO	EVENT_ID	YEAR	visit_date	subgroup	COHORT	age_at_visit	td_pigd	pigd	hy
100001	BL	0	10/2020	Sporadic PD	1	67.386301	1.0	0.4	2.0
100001	V04	1	11/2021	Sporadic PD	1	68.512329	1.0	0.4	2.0
100001	V06	2	11/2022	Sporadic PD	1	69.528767	1.0	0.4	2.0

Read by `loaders.load_curated_cut()` (lines 45–48).

Columns used: `PATNO`, `EVENT_ID` (merge keys), `subgroup` (cohort filter), `YEAR` and `visit_date` (time axis), `td_pigd` (validation reference only), plus the baseline predictor columns written to `baseline.csv`.

This file already contains PPMI's own derived scores (`pigd`, `hy`, `updrs3_score`, `td_pigd`). **The pipeline does not use any of them to build the label.** It uses this file purely as a visit skeleton plus covariates, and computes the subtype independently from raw items. That independence is what makes the validation in Section 4.5 meaningful — otherwise it would be comparing a number against itself.

File 2 — patient questionnaire items

`data/motor_updrs/MDS_UPDRS_Part_II__Patient_Questionnaire_20Aug2026.csv`

Shape	**37,777 rows × 22 columns**
Unique `PATNO`	5,341
Duplicate `(PATNO, EVENT_ID)`	**0**

One row = one patient's self-reported questionnaire at one visit.

All 22 columns: `REC_ID, PATNO, EVENT_ID, PAG_NAME, INFODT, NUPSOURC, NP2SPCH, NP2SALV, NP2SWAL, NP2EAT, NP2DRES, NP2HYGN, NP2HWRT, NP2HOBB, NP2TURN, NP2TRMR, NP2RISE, NP2WALK, NP2FREZ, NP2PTOT, ORIG_ENTRY, LAST_UPDATE`.

The pipeline reads exactly three item columns: `NP2TRMR` (tremor), `NP2WALK` (walking and balance), `NP2FREZ` (freezing). The other ten items are not used. Read by `loaders.load_updrs_part2()`.

File 3 — motor examination items

`data/motor_updrs/MDS-UPDRS_Part_III_20Aug2026.csv`

Shape	**39,310 rows × 65 columns**
Unique `PATNO`	5,344
Duplicate `(PATNO, EVENT_ID)`	**6,907**

One row = one clinician-administered motor examination — not one row per visit.

The most important structural fact in the raw data. Part II has zero duplicate `(PATNO, EVENT_ID)` pairs; Part III has 6,907. The reason is in the columns: `PDSTATE` (`ON`/`OFF`), `PDTRTMNT`, `ONEXAM`, `OFFEXAM`, `ONOFFORDER`. A medicated patient can be examined *twice at the same visit* — once off medication and once on it — producing two rows with different scores:

```
PATNO   EVENT_ID  PDSTATE  PDTRTMNT  NP3GAIT  NP3PSTBL
3002    V06       OFF      1         2        1          <- exam A
3002    V06       ON       1         1        0          <- exam B, same visit
```

Merging this against the visit skeleton without handling it would either duplicate the visit or silently keep whichever row happened to come first. Both are wrong. This is precisely why `select_medication_state()` and `deduplicate_visits()` exist.

The pipeline reads thirteen item columns plus three metadata columns: `NP3PTRMR, NP3PTRML, NP3KTRMR, NP3KTRML, NP3RTARU, NP3RTALU, NP3RTARL, NP3RTALL, NP3RTALJ, NP3RTCON` (tremor), `NP3GAIT, NP3FRZGT, NP3PSTBL` (gait and postural), plus `PDSTATE`, `PDTRTMNT`, `NHY`. The remaining ≈ 46 columns are not used.

File 4 — the validation reference

`data/validation/PPMI_Original_Cohort_BL_to_Year_5_Dataset_Apr2020.csv`

Shape				**3,445 rows × 159 columns**
Unique `PATNO`				683
`td_pigd_old`	1.0: 1,440	2.0: 550	3.0: 1,014	NaN: 441

This file is **never used for analysis**. It is read once, by `validate.against_archived_three_class()`, purely to check the recomputed labels. It is the only file in the repository carrying a **three-class** subtype label, because the current curated cut collapsed its `td_pigd` to two classes.

3.2 Why these sixteen items?

What the code does. `config.yaml` lines 42–60 list eleven tremor items and five PIGD items.

Documented rationale, quoted verbatim from `config.yaml` lines 36–39: *"Definition follows Stebbins et al., Movement Disorders 2013, and matches the derivation notes in the Curated Data Cut dictionary for `td_pigd` verbatim."*

Interpretation. The item selection is not a modelling choice this project made; it is a published clinical definition that was cross-checked against PPMI's own derivation notes.

Assumption introduced. That the Stebbins definition is the right phenotype to model. The proposal commits to this explicitly — its stated goal is to *retain* the clinically validated ratio rather than invent a new score.

4 Stage 1 — `01_build_panel.py`

4.1 A. Purpose

Scientific question: *For every visit of every sporadic PD patient in PPMI, what was that patient's motor subtype at that moment, and can we prove the label is correct?*

This script converts raw MDS-UPDRS item scores into a longitudinal, labelled patient-visit panel, and validates that panel against PPMI's own derivations before any analysis is permitted to use it.

4.2 B. Inputs

Three files (Section 3): the curated cut, MDS-UPDRS Part II, MDS-UPDRS Part III. A fourth, the archived cohort, is read only for validation.

4.3 C. Code walkthrough with actual counts

The script's real work is one call, `build_visit_panel(cfg)` in `panel.py` line 56, which decomposes as follows.

Step 1.1 — `filter_cohort()` (`panel.py`:44)

```
out = frame[frame["subgroup"].isin(keep)]
out = out[~out["EVENT_ID"].isin(set(cohort["excluded_events"]))]
out = out[out["YEAR"] <= cohort["max_follow_up_years"]]
```

What happened to the removed rows. The 13,661 rows dropped at the first step are the other PPMI subgroups: Hyposmia (1,759 patients), RBD (572), LRRK2 (400), Healthy Control (330), GBA (310), SWEDD (60) and others. They are not deleted from disk; they are simply not selected. The 15 further rows are visits labelled `ST` (unscheduled) or `PW` (premature withdrawal) — verified:

Operation	Rows	Removed
`load_curated_cut()`	19,450	—
`subgroup` $\in$ {Sporadic PD}	5,789	13,661
`EVENT_ID` $\notin$ {ST, PW, U01, U02}	5,774	15
`YEAR` ≤ 15	5,774	0

Table 2: Cohort filtering. Patients: 4,788 → 1,080.

those are the only two excluded codes actually present. The `YEAR` filter removes nothing, because the curated cut already stops at year 15.

Why this step exists. `config.yaml` lines 114–116 document it as *"Sporadic PD is the primary analysis population; the genetic PD subgroups are listed so a sensitivity analysis can switch them on without touching code."*

The scientific rationale for restricting to sporadic PD is not explicitly documented in the repository. A defensible interpretation — label it as interpretation, not as a recorded decision — is that genetic PD subgroups (LRRK2, GBA, SNCA) have known differences in motor phenotype and progression rate, so pooling them would mix populations. The config is written so this can be tested rather than assumed.

Step 1.2 — `select_medication_state()` (`subtype.py`:159)

```
state = frame[state_col].astype("string").str.strip().str.upper()
matches = state == cfg.medication_state          # "OFF"
if treat_col in frame.columns:
    untreated = pd.to_numeric(frame[treat_col], errors="coerce") == 0
    matches = matches | untreated                # keep untreated participants
matches = matches | state.isna()                 # keep rows with no PDSTATE
return frame[matches.fillna(False)].copy()
```

Operation	Rows	Removed
Part III raw	39,310	—
After `select_medication_state`	29,356	9,954

Verified: all 9,954 removed rows have `PDSTATE == 'ON'`. Not a single row with `PDSTATE == 'OFF'`, with a missing `PDSTATE`, or from an untreated participant was discarded. The filter does exactly what it claims.

Why this exists. Levodopa suppresses tremor. Scoring an on-medication examination would shift a patient's ratio toward PIGD and manufacture transitions that are pharmacological rather than pathological. The canonical Stebbins classification is defined on the OFF or untreated exam.

Assumption introduced. That an OFF exam reflects the underlying untreated phenotype. In practice "OFF" in PPMI means a practically-defined off state (typically overnight withdrawal), not a true drug-free state.

Step 1.3 — merge Part II with Part III (`panel.py`:76)

```
merged = part2[KEY + part2_items].merge(
    part3[KEY + part3_items + carry], on=KEY, how="outer"
)
```

An *outer* join is used, so a visit with only a questionnaire or only an exam is retained rather than silently dropped; it will simply fail the completeness test later and produce no label. Eleven duplicate keys survive the medication filter — these are visits with more than one qualifying OFF-state record.

Operation	Rows	Duplicate keys
Part II (3 items) ⋈ Part III OFF (13 items + 3 carried)	37,897	11

Step 1.4 — `deduplicate_visits()` (`subtype.py`:**187**)

```
present = tuple(c for c in cfg.all_items if c in frame.columns)
cleaned = clean_items(frame, present, cfg)
ordered = frame.assign(_n_missing=cleaned.isna().sum(axis=1))
ordered = ordered.sort_values(["PATNO", "EVENT_ID", "_n_missing"], kind="mergesort")
deduped = ordered.drop_duplicates(["PATNO", "EVENT_ID"], keep="first")
```

Operation	Rows	Removed
After merge	37,897	—
After deduplication	37,886	11

The tie-break rule: keep the row with the *fewest missing items*, counted *after* sentinel cleaning, and break remaining ties by first occurrence using a stable sort (`kind="mergesort"`). The result does not depend on the order pandas happened to read the file. This is unit-tested in `tests/test_subtype.py::TestDeduplication`.

Step 1.5 — `compute_subscores()` (`subtype.py`:**77**)

Covered in full in Section 5. Applied to all 37,886 patient-visits in the merged frame:

`tremor_score` non-null	27,046
`pigd_score` non-null	24,098
`subtype` non-null	**24,066**
`ratio_defined` True	12,644

Note that subscores are computed for the *entire PPMI population*, not only the analysis cohort. The restriction to sporadic PD happens at the next step, via the join.

Step 1.6 — the join onto the visit skeleton (`panel.py`:**69**)

```
panel = curated.merge(subscores, on=KEY, how="left", validate="one_to_one")
```

The direction of this join is a design decision worth defending. It is a *left* join from the curated cut, so the row count is fixed by the visit skeleton, not by the item files. A patient-visit exists in the panel if and only if PPMI's curated cut says the visit happened. Item data can only *fill in* a label, never add or remove a row. The `validate="one_to_one"` argument makes pandas raise if either side has duplicate keys — an assertion that the deduplication actually worked.

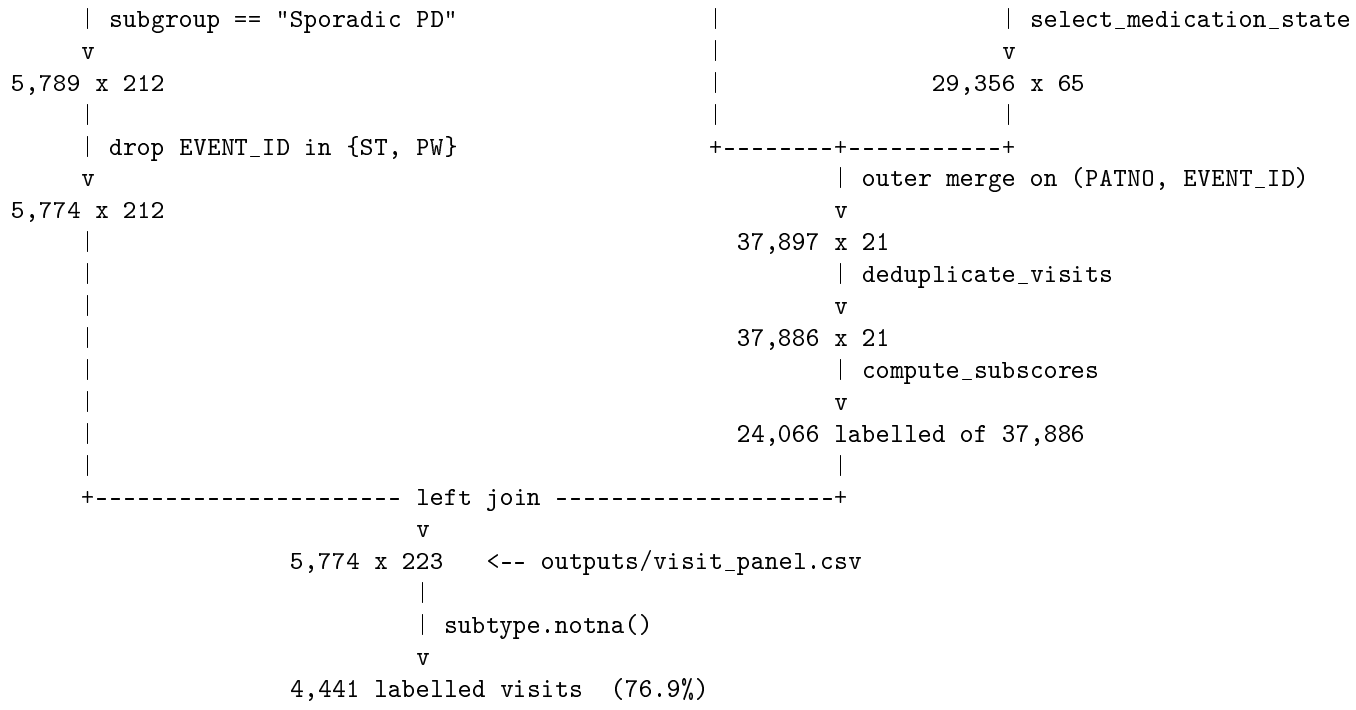
Step 1.7 — `_add_time_axis()` (`panel.py`:**97**)

Covered in Section 6.

4.4 D. Data shape after each step

```
CURATED CUT                              MDS-UPDRS PART II    MDS-UPDRS PART III
19,450 x 212                             37,777 x 22          39,310 x 65
     |                                        |                    |
```

Left side (cohort visits)	5,774
Right side (all subscores)	37,886
Result	**5,774 × 223**
of which labelled	**4,441** (76.9%)



4.5 E. Validation (`validate.py`)

Two independent checks run before the panel is written.

Reference	Classes	Agreement
Curated cut `td_pigd`	2 (TD / non-TD)	4,431 / 4,431 = **100.00%**
Archived `td_pigd_old`	3 (TD / Indet. / PIGD)	1,595 / 1,597 = **99.87%**

The two-class check confirms the arithmetic, the item lists, the medication filter and the 1.15 threshold. It cannot test the Indeterminate/PIGD boundary, because the curated cut collapses those classes. The three-class check, over 390 overlapping patients, is the only test of that boundary.

A trap that was hit and fixed. The archived `td_pigd_old` codes **2 = PIGD and 3 = Indeterminate**, the reverse of this project's coding. Compared without remapping, agreement reads 65.9% and looks like a broken classifier. `validate.remap_archived_labels()` (line 68) applies the correction, which is driven by `config.yaml` lines 80–83 and unit-tested in `tests/test_validate.py`. The remap is verified against the archived data dictionary, not guessed.

4.6 F. Research reasoning

What the code does. Restricts to a homogeneous cohort, selects the clinically correct examination, deduplicates deterministically, computes a published phenotype from raw items, and proves the result against two independent references.

Why the researcher chose to do it. The proposal requires a *three-class* label and a *continuous ratio* at every visit. PPMI's curated cut supplies neither: it has no tremor subscore, and its `td_pigd` is two-class. Recomputation was not optional.

Assumptions introduced. (i) Stebbins 2013 is the right phenotype; (ii) an OFF exam represents the untreated state; (iii) sporadic PD is the appropriate population; (iv) a visit with any missing item yields no label.

What could go wrong. Assumption (iv) is the one that bites: it discards 1,333 of 5,774 visits, and not at random. This is examined in question Q6 of Section 12.

5 Target / label construction

5.1 The formula, exactly as implemented

$$\text{tremor_score} = \frac{1}{11}\sum_{i\in\mathcal{T}} x_i, \qquad \text{pigd_score} = \frac{1}{5}\sum_{j\in\mathcal{P}} x_j, \qquad \text{ratio} = \frac{\text{tremor_score}}{\text{pigd_score}}$$

where $\mathcal{T}$ is the eleven tremor items and $\mathcal{P}$ the five gait/postural items. Both are **means, not sums**.

$$\text{subtype} = \begin{cases} \text{TD} & \text{ratio} \geq 1.15 \\ \text{TD} & \text{pigd_score} = 0 \ \wedge \ \text{tremor_score} > 0 \\ \text{Indeterminate} & 0.90 < \text{ratio} < 1.15 \\ \text{Indeterminate} & \text{tremor_score} = 0 \ \wedge \ \text{pigd_score} = 0 \\ \text{PIGD} & \text{ratio} \leq 0.90 \end{cases}$$

Codes are `TD = 1`, `Indeterminate = 2`, `PIGD = 3` (`config.yaml` lines 71–74).

5.2 Step 1 — cleaning (`clean_items`, `subtype.py`:39)

```
lo, hi = cfg.valid_score_range                    # (0, 4)
block = frame[list(items)].apply(pd.to_numeric, errors="coerce")
return block.where((block >= lo) & (block <= hi))
```

PPMI stores "Unable to Rate" as the value 101, in the same numeric field as a real 0–4 rating, in all thirteen Part III items. Averaged as a score, a single 101 among the five PIGD items lifts their mean to roughly 20, drives the ratio to almost zero, and relabels a tremor-dominant visit as PIGD. `NP3PSTBL` alone carries 3,206 such codes among OFF exams.
The implementation excludes them by *range*, not by an equality test against 101, so a sentinel PPMI introduces later cannot silently enter a subscore. Six unit tests in `tests/test_subtype.py::TestUnrateableCodes` pin this down.

5.3 Step 2 — subscores with a missingness policy (`_subscore`, line 62)

```
block = clean_items(frame, items, cfg)
n_missing = block.isna().sum(axis=1)
score = block.mean(axis=1, skipna=True)
return score.where(n_missing <= max_missing)       # max_missing = 0
```

With `max_missing_tremor_items: 0` and `max_missing_pigd_items: 0`, **any** missing constituent nullifies the subscore. This matches PPMI's own rule and is why the two-class validation reaches exactly 100%.

5.4 Step 3 — classification (`classify`, line 121)

```
both_zero      = observed & (t == 0) & (p == 0)
pigd_zero_only = observed & (p == 0) & (t > 0)
divisible      = observed & (p != 0)
```

```
ratio = t / p.where(p != 0)

result[divisible & (ratio >= cfg.td_threshold)]   = labels["TD"]
result[divisible & (ratio <= cfg.pigd_threshold)] = labels["PIGD"]
result[divisible & (ratio > pigd_thr) & (ratio < td_thr)] = labels["Indeterminate"]

result[pigd_zero_only] = labels["TD"]              # applied last: cannot be overwritten
result[both_zero]      = labels["Indeterminate"]
```

The degenerate cases are handled by explicit masks applied *after* the ratio cases, so they cannot be overwritten by a NaN comparison. A property test (`test_every_observed_visit_is_classified`) sweeps a grid of 17×17 score pairs and asserts no observed pair ever yields NaN.

5.5 Edge cases, and how each is resolved

Situation	Naive result	Implemented result
pigd $= 0$, tremor > 0	∞ or crash	TD
tremor $= 0$, pigd $= 0$	NaN (0/0)	Indeterminate
tremor $= 0$, pigd > 0	ratio $= 0$	PIGD (via ≤ 0.90)
ratio exactly 1.15	ambiguous	TD (boundary closed)
ratio exactly 0.90	ambiguous	PIGD (boundary closed)
any item missing	biased mean	no label
item $= 101$	mean ≈ 20	treated as missing

5.6 A fully worked example from the real data

PATNO 3010, visit BL. Raw items as they appear on disk:

```
Part II:   NP2TRMR = 1     NP2WALK = 1     NP2FREZ = 0
Part III:  PDSTATE = NaN   PDTRTMNT = 0            <-- untreated participant
           NP3PTRMR = 1  NP3PTRML = 1  NP3KTRMR = 1  NP3KTRML = 1
           NP3RTARU = 0  NP3RTALU = 0  NP3RTARL = 0  NP3RTALL = 0
           NP3RTALJ = 0  NP3RTCON = 0
           NP3GAIT  = 0  NP3FRZGT = 0  NP3PSTBL = 0
```

Note first that `PDSTATE` is missing and `PDTRTMNT = 0`. This participant was not on PD medication at baseline, so there is no ON/OFF distinction to make. `select_medication_state()` keeps the row through its `untreated` clause — exactly the case its `test_untreated_participants_are_kept` test covers.

Tremor score. Eleven items, values $1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0$:

$$\text{tremor_score} = \frac{1+1+1+1+1+0+0+0+0+0+0}{11} = \frac{5}{11} = 0.4545$$

PIGD score. Five items, values $1, 0, 0, 0, 0$:

$$\text{pigd_score} = \frac{1+0+0+0+0}{5} = \frac{1}{5} = 0.2000$$

Ratio.

$$\text{ratio} = \frac{0.4545}{0.2000} = 2.2727$$

Classification. $2.2727 \geq 1.15 \Rightarrow$ **TD (code 1)**.

This matches `outputs/visit_panel.csv` exactly: `tremor_score = 0.454545`, `pigd_score = 0.2`, `ratio = 2.272727`, `subtype_name = TD`.

5.7 The ratio is not always defined

Of 4,441 labelled visits, **725 (16.3%)** have pigd_score = 0. Those visits carry a valid *label* (the rule sends them to TD) but no finite *ratio*. `compute_subscores` records this distinction explicitly:

```
out["ratio"]         = tremor / pigd.replace(0.0, np.nan)   # NaN, not inf
out["ratio_defined"] = both_observed & (pigd != 0)
```

This matters because the proposed hierarchical model consumes the *continuous ratio* as a noisy observation of a latent phenotype. For one visit in six there is nothing to consume. The flag exists so that population can be counted and handled deliberately rather than dropped silently.

6 Longitudinal panel construction

6.1 The time axis (`_add_time_axis`, `panel.py`:97)

```
out["visit_ts"] = parse_visit_date(out["visit_date"])          # "%m/%Y"
out = out.sort_values(["PATNO", "visit_ts", "YEAR"], kind="mergesort")
baseline = out.groupby("PATNO")["visit_ts"].transform("min")
out["years_from_baseline"] = (out["visit_ts"] - baseline).dt.days / 365.25
out["years_from_baseline"] = out["years_from_baseline"].fillna(out["YEAR"].astype(float))
out["visit_index"] = out.groupby("PATNO").cumcount()
out["n_visits"]    = out.groupby("PATNO")["PATNO"].transform("size")
```

Visit identity
`(PATNO, EVENT_ID)`. `EVENT_ID` is a label such as `BL`, `V04`, `V06`, not a time.

Baseline Each patient's *own earliest* visit date (`transform("min")`), not a fixed calendar date and not necessarily `EVENT_ID == "BL"`.

Elapsed time
(visit date − baseline date)/365.25 days.

Fallback If a date is unparseable, the nominal `YEAR` is used, so a missing `INFODT` does not drop a visit from the time axis.

Sorting Stable mergesort, so ties resolve reproducibly.

Why actual dates and not `YEAR`? `YEAR` is derived from `EVENT_ID` — it is a *visit label*, not elapsed time. Patients miss visits and attend late. In the panel, `V04` occurs anywhere from 0.83 to over 1.4 years after baseline. Using `YEAR` would assign every `BL`→`V04` pair an interval of exactly 1.0, which is false, and would make the interval-stratified analysis in Section 8 meaningless.
Limitation, stated: PPMI releases `INFODT` as month and year only, so every date is anchored to the first of the month. Intervals carry roughly ± two weeks of noise — negligible at annual spacing.

6.2 What one patient's trajectory looks like

PATNO 3010, taken verbatim from `outputs/visit_panel.csv`:

EVENT_ID	YEAR	visit_date	years_from_baseline	tremor	pigd	ratio	subtype	
BL	0	06/2011	0.000000	0.4545	0.2	2.2727	TD	
V04	1	06/2012	1.002053	0.9091	0.2	4.5455	TD	
V06	2	07/2013	2.083504	0.8182	0.2	4.0909	TD	
V08	3	09/2014	3.252567	NaN	NaN	NaN	---	<- unlabelled
V10	4	06/2015	4.000000	1.6364	0.2	8.1818	TD	
V12	5	11/2016	5.420945	NaN	NaN	NaN	---	<- unlabelled
V14	7	08/2018	7.167693	0.2727	0.6	0.4545	PIGD	
V15	8	07/2019	8.082136	1.4545	0.8	1.8182	TD	
V17	10	05/2021	9.916496	NaN	NaN	NaN	---	
V18	11	06/2022	11.000684	NaN	NaN	NaN	---	

Observe: V06 is at 2.083 years, not 2.000. V17 is at 9.92 years despite being labelled year 10. V12 is at 5.42 years. The nominal year and the elapsed time genuinely differ.

6.3 Patients with too few usable visits

Patients with ≥ 1 labelled visit	1,080
Patients with exactly 1 labelled visit	**146**
Patients with ≥ 2 labelled visits	**934**

The 146 single-visit patients remain in visit_panel.csv — they are not deleted — but they cannot contribute a transition, so they contribute nothing to Stage 2. restrict_to_analysable() (panel.py:120) applies the min_visits_per_patient: 2 rule when producing outputs/baseline.csv, which therefore has **934 rows**.

7 Transition construction

7.1 consecutive_pairs() (transitions.py:16)

```
df = panel[panel[label_col].notna()].sort_values(["PATNO", time_col])
grouped = df.groupby("PATNO", sort=False)

out = pd.DataFrame({
    "PATNO":      df["PATNO"],
    "from_state": df[label_col],
    "to_state":   grouped[label_col].shift(-1),
    "from_time":  df[time_col],
    "to_time":    grouped[time_col].shift(-1),
    "from_event": df["EVENT_ID"],
    "to_event":   grouped["EVENT_ID"].shift(-1),
})
out = out[out["to_state"].notna()].copy()
out["interval_years"] = out["to_time"] - out["from_time"]
```

Two decisions are encoded here:

1. **Unlabelled visits are filtered out *before* pairing** (line 1). They therefore *bridge* rather than truncate: a patient labelled at years 0 and 2 but not 1 yields one pair with a two-year interval, not a broken sequence.
2. **shift(-1) is applied within groupby("PATNO")**, so the last visit of one patient can never pair with the first visit of the next. tests/test_transitions.py::test_pairs_are_within_patient asserts this.

7.2 The same patient, transformed

PATNO 3010 — unlabelled visits removed, then paired:

```
from_event to_event  from_state  to_state  interval_years
        BL      V04          TD        TD        1.002053
       V04      V06          TD        TD        1.081451
       V06      V10          TD        TD        1.916496   <- V08 was unlabelled: BRIDGED
       V10      V14          TD      PIGD        3.167693   <- V12 unlabelled: 3.17y interval
       V14      V15        PIGD        TD        0.914442   <- an immediate REVERSION
```

This single patient demonstrates three mechanisms at once: gap bridging (V06→V10), a long-interval transition (V10→V14, 3.17 years, not 3.0), and a state change that reverses at the very next visit (V14→V15). The last is exactly what the reversion analysis in Section 8 counts.

7.3 Arithmetic check on the pair count

A patient with n_i labelled visits contributes $n_i - 1$ pairs. Summed over the 934 patients with $n_i \geq 2$:

$$\sum_i (n_i - 1) = 3{,}361$$

which is exactly the number of rows `consecutive_pairs()` returns. **Verified independently.** No pair is lost or double-counted.

7.4 From counts to probabilities (`transition_matrix`, line 47)

```
counts = pd.crosstab(pairs["from_state"], pairs["to_state"])
counts = counts.reindex(index=states, columns=states, fill_value=0)
totals = counts.sum(axis=1)
probs  = counts.div(totals.where(totals > 0), axis=0)
```

Raw counts, before normalisation (total = 3,361):

	→ TD	→ Ind	→ PIGD	Row total
TD	1827	181	253	2261
Indeterminate	147	64	123	334
PIGD	141	87	538	766

After dividing each row by its total:

	→ TD	→ Ind	→ PIGD
TD	0.808	0.080	0.112
Indeterminate	0.440	0.192	0.368
PIGD	0.184	0.114	0.702

Worked entry: $181/2261 = 0.0800$. The `reindex` guarantees all three states appear even if one were unobserved, and `totals.where(totals > 0)` prevents a division by zero for an empty row.

8 Statistical analysis

8.1 Wilson score intervals (`transition_counts_with_ci`, line 67)

Question answered. How precise is each transition probability?

Formula, as implemented:

$$\hat{p} = \frac{k}{n}, \qquad \text{centre} = \frac{\hat{p} + \frac{z^2}{2n}}{1 + \frac{z^2}{n}}, \qquad \text{half-width} = \frac{z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

with $z = \Phi^{-1}(1 - \alpha/2)$, $\alpha = 0.05$.

Why not the normal approximation? Row counts range from 334 to 2,261, and several probabilities sit near 0 or 1 (`TD`→`Ind` is 0.080). The Wald interval $\hat{p} \pm z\sqrt{\hat{p}(1-\hat{p})/n}$ produces bounds outside $[0, 1]$ in exactly those cells. A unit test (`test_wilson_interval_stays_in_unit_range`) asserts the implementation never does.

Interpretation. `TD`→`TD` is 0.808, CI $[0.791, 0.824]$. **Incorrect interpretation:** treating these as independent — the three probabilities in a row are constrained to sum to 1, so their errors are negatively correlated.

8.2 Interval stratification (line 106)

Question answered. Does a transition probability mean anything without saying over what period?

Bins: $[0, 1.5)$, $[1.5, 2.5)$, $[2.5, 4.0)$, $[4.0, \infty)$ years.

Interval (years)	Pairs	TD → PIGD
≤ 1.5	2,882	0.100
1.5–2.5	302	0.168
2.5–4.0	111	0.153
> 4.0	66	0.289

Conclusion this forces. The pooled matrix is not a well-defined quantity — it averages one-year and multi-year transitions in whatever proportion attendance happened to produce. It describes the study design as much as the disease.

8.3 Reversion analysis (`reversion_after_change`, line 198)

Question answered. Of the state changes we observe, how many are real?

```
frame = pd.DataFrame({
    "s0": df[label_col],
    "s1": g[label_col].shift(-1),
    "s2": g[label_col].shift(-2),
    ...})
frame = frame[frame["s2"].notna() & (frame["s0"] != frame["s1"])]
frame["reverted"] = frame["s2"] == frame["s0"]
```

It takes *triples*: a change $s_0 \neq s_1$ that has a third labelled visit s_2, and asks whether $s_2 = s_0$.

Qualifying changes (denominator)	673
Of which reverted	237
Reversion rate	**0.3522**
Patients contributing	342

Assumption. That a change followed by a return is more likely measurement noise than two genuine changes. **Incorrect interpretation:** concluding the labels are worthless. Section 4.3 of the Weeks 3–4 report shows the movement is significantly *asymmetric* (TD→PIGD 253 versus PIGD→TD 141, $p = 1.8 \times 10^{-8}$), so a real directional drift exists beneath the churn.

Limitation. Requires three labelled visits, so the estimate comes from the better-attended subset — 342 of 934 patients.

8.4 Kaplan–Meier (`first_entry` + `kaplan_meier`, lines 246, 299)

Question answered. How long until a patient first becomes PIGD?

Formula, as implemented:

$$\hat{S}(t) = \prod_{t_i \leq t} \left(1 - \frac{d_i}{n_i}\right), \qquad \widehat{\mathrm{SE}}(\hat{S}(t)) = \hat{S}(t) \sqrt{\sum_{t_i \leq t} \frac{d_i}{n_i(n_i - d_i)}}$$

(Greenwood's formula). Cumulative incidence is plotted as $1 - \hat{S}(t)$.

Risk set construction. `first_entry()` returns every patient with a `converted` flag and a time; patients already PIGD at their first visit are flagged `starts_in_state` and excluded.

	Naive (1 visit)	Sustained (2 visits)
Patients	1,080	1,080
Excluded (PIGD at baseline)	195	195
At risk	**885**	**885**
Converted	263	127
KM cumulative incidence, yr 5	**35.25%**	**15.33%**
n at risk at year 5	180	211

Why Kaplan–Meier and not a percentage? Follow-up ranges from one visit to fourteen years. A raw proportion conflates "did not convert" with "was not observed long enough". KM censors at the last labelled visit.

The `min_consecutive` parameter. With `min_consecutive=2`, an entry counts only if the patient is still PIGD at the next labelled visit. Requiring this removes **136 of 263** apparent conversions — 51.7%.

Assumption. Non-informative censoring: that dropping out is unrelated to subtype. **This is questionable here** and is flagged as a limitation.

8.5 Pathway analysis (`state_sequences` + `collapse_runs`, lines 177, 185)

```
def collapse_runs(sequence):
    out = []
    for s in sequence:
        if not out or out[-1] != s:
            out.append(s)
    return tuple(out)
```

`(TD, TD, PIGD, PIGD)` becomes `(TD, PIGD)`. **Why:** without collapsing, pathway frequencies would be dominated by how often a patient attended rather than by what happened to them.

Patients with ≥ 1 label	1,080	
Distinct collapsed pathways	86	
Never changed state	622	57.6%

9 Trace of every major result

Each trace follows the form RESULT → SCRIPT → FUNCTION → DATAFRAME → OPERATION → NUMBER.

19,450 visits

```
loaders.load_curated_cut(cfg)
  -> pd.read_excel("data/clinical/PPMI_Curated_Data_Cut_Public_20260511.xlsx",
                   sheet_name="20260511")
  -> df.shape = (19450, 212)
```

5,774 cohort visits

```
01_build_panel.py -> build_visit_panel() -> panel.filter_cohort()
  frame[frame["subgroup"].isin({"Sporadic PD"})]           19,450 -> 5,789
  frame[~frame["EVENT_ID"].isin({ST, PW, U01, U02})]        5,789 -> 5,774
  frame[frame["YEAR"] <= 15]                                5,774 -> 5,774
  SAVED: outputs/visit_panel.csv  (5,774 rows x 223 cols)
```

29,356 OFF-state exam rows

```
panel.build_subscores() -> subtype.select_medication_state()
  keep if PDSTATE == "OFF"  OR  PDTRTMNT == 0  OR  PDSTATE is NaN
  39,310 -> 29,356    (9,954 removed, all PDSTATE == "ON")
```

4,441 labelled visits

```
panel.build_visit_panel()
  curated(5,774) .merge(subscores, how="left", validate="one_to_one")  -> 5,774
  panel["subtype"].notna().sum()                                       -> 4,441
  = 76.9% of cohort visits
  SAVED: outputs/visit_panel.csv, column `subtype`
  REPORTED: outputs/panel_summary.txt
```

934 patients / 146 excluded

```
labelled = panel[panel.subtype.notna()]
per_patient = labelled.groupby("PATNO").size()
  (per_patient == 1).sum()  -> 146     cannot form a transition
  (per_patient >= 2).sum()  -> 934
  SAVED: outputs/baseline.csv  (934 rows, via panel.restrict_to_analysable)
```

3,361 transition pairs

```
02_transition_analysis.py -> transitions.consecutive_pairs(panel)
  df = panel[panel["subtype"].notna()].sort_values(["PATNO","years_from_baseline"])
  to_state = df.groupby("PATNO")["subtype"].shift(-1)
  out = out[out["to_state"].notna()]
  len(out) -> 3,361      CHECK: sum(n_i - 1) over 934 patients = 3,361
```

0.808 (TD → TD)

```
transitions.transition_matrix(pairs, states=[1,2,3])
  counts = pd.crosstab(pairs["from_state"], pairs["to_state"])
  counts.loc[TD, TD]  = 1827
  counts.loc[TD].sum()= 2261
  1827 / 2261 = 0.8080
  SAVED: outputs/transitions/transition_matrix_probs.csv
```

35.2% reversion

```
transitions.reversion_after_change(panel)
  triples (s0, s1, s2) where s2 is not NaN AND s0 != s1     -> 673 rows
  reverted = (s2 == s0)                                     -> 237 True
  237 / 673 = 0.3522
  SAVED: outputs/transitions/reversion_summary.csv
```

35.3% vs 15.3% conversion at year 5

```
transitions.first_entry(panel, target_state=3, min_consecutive=k)
  at_risk = fe[~fe["starts_in_state"]]                       -> 885 patients
  km = transitions.kaplan_meier(at_risk.time, at_risk.converted)
  1 - km[km.time <= 5.0].survival.iloc[-1]
      k=1 -> 0.3525   (n at risk 180)
      k=2 -> 0.1533   (n at risk 211)
  SAVED: outputs/transitions/conversion_km_{naive,sustained}.csv
```

100.00% and 99.87% validation

```
01_build_panel.py -> validate.against_curated_cut(panel, cfg)
  ours_binary = collapse_to_binary(panel["subtype"])   # TD->1, else 2
  compare vs panel["td_pigd"]        -> 4,431 compared, 4,431 agree
01_build_panel.py -> validate.against_archived_three_class(panel, cfg)
  ref = archived[["PATNO","EVENT_ID","td_pigd_old"]]
  ref = remap_archived_labels(ref)   # 2<->3 swap, per archived dictionary
  inner join, compare               -> 1,597 compared, 1,595 agree
  SAVED: outputs/validation.txt
```

10 Complete data funnel

```
PPMI CURATED DATA CUT                                        19,450 visits (4,788 patients)
     |
     |  subgroup == "Sporadic PD"        -- other cohorts (Hyposmia, RBD, HC,
     |                                      LRRK2, GBA, SWEDD...) not selected
     v
                                                              5,789 visits (1,080 patients)
     |  drop EVENT_ID in {ST, PW}        -- unscheduled / premature-withdrawal visits
     v
COHORT VISIT SKELETON                                         5,774 visits (1,080 patients)
     |
     |  ---------------- meanwhile, on the item files ----------------
     |
     |   MDS-UPDRS Part II   37,777 rows   (0 duplicate visit keys)
     |   MDS-UPDRS Part III  39,310 rows   (6,907 duplicate visit keys)
     |         |
     |         |  select_medication_state: keep OFF / untreated
     |         v
     |   Part III OFF        29,356 rows   (9,954 ON-state rows removed)
     |         |
     |         |  outer merge on (PATNO, EVENT_ID)
     |         v
     |                       37,897 rows   (11 duplicate keys remain)
     |         |
     |         |  deduplicate_visits: fewest missing items wins
     |         v
     |                       37,886 patient-visits
     |         |
     |         |  clean_items (drop 101 / out-of-range) -> subscores -> classify
     |         v
     |                       24,066 labelled  (of 37,886, all PPMI patients)
     |         |
     +---------+  LEFT JOIN onto the cohort skeleton
     v
LABELLED PANEL                                    5,774 rows x 223 cols
                                                  outputs/visit_panel.csv
     |
     |  subtype.notna()      -- 1,333 visits have no complete OFF-state item set
     v
                                                  4,441 labelled visits (76.9%)
     |
     |  group by PATNO, sort by years_from_baseline
     v
                                                  1,080 patients
     |  keep patients with >= 2 labelled visits    -- 146 single-visit patients
     v                                               cannot form a transition
                                                  934 patients
```

```
     |
     |  consecutive_pairs: shift(-1) within patient
     v
TRANSITION PAIRS                                    3,361 pairs
     |
     +--> transition_matrix        -> 3x3 counts and probabilities
     +--> transition_..._with_ci   -> Wilson 95% intervals
     +--> ..._by_interval          -> 4 interval strata (2,882 / 302 / 111 / 66)
     +--> reversion_after_change   -> 673 changes, 237 reverted = 35.2%
     +--> first_entry + kaplan_meier -> 885 at risk, 263 / 127 converted
     +--> state_sequences          -> 86 distinct pathways, 622 never changed
     v
FINAL RESULTS                                       outputs/transitions/
                                                    9 CSV tables, 6 figures, report.txt
```

11 Code ↔ science map

All modules live in `src/ppmi_subtype/`.

Concept	Module	Function	Operation	Output
Study population	`panel`	`filter_cohort`	subgroup + event filter	5,774 visits
Correct exam state	`subtype`	`select_medication_state`	keep OFF or untreated	29,356 rows
One record per visit	`subtype`	`deduplicate_visits`	fewest-missing tie-break	37,886 rows
Invalid measurements	`subtype`	`clean_items`	drop outside 0–4	cleaned items
Motor phenotype	`subtype`	`compute_subscores`	two means + ratio	tremor, pigd, ratio
Clinical subtype	`subtype`	`classify`	1.15 / 0.90 thresholds	TD / Ind / PIGD
Label correctness	`validate`	`against_curated_cut`, `against_archived_three_class`	agreement	100% and 99.87%
Disease timeline	`panel`	`_add_time_axis`	date − patient min	`years_from_baseline`
Patient trajectory	`transitions`	`state_sequences`	groupby + tuple	state sequence
Observed transition	`transitions`	`consecutive_pairs`	groupby + shift(−1)	3,361 pairs
Progression rates	`transitions`	`transition_matrix`	crosstab + row-normalise	3×3 matrix
Estimate precision	`transitions`	`transition_counts_with_ci`	Wilson score	95% CIs
Time-dependence	`transitions`	`transition_matrix_by_interval`	cut + per-bin matrix	4 matrices
Measurement noise	`transitions`	`reversion_after_change`	triples s_0, s_1, s_2	35.2%
Time to event	`transitions`	`first_entry`, `kaplan_meier`	survival + censoring	KM curves
Common courses	`transitions`	`collapse_runs`	drop repeated states	86 pathways

Concept	Module	Function	Operation	Output
Figures	`plots`	six plotting functions	matplotlib	6 PNG files

12 Professor defence questions

Q1. Why did you recompute the label instead of using PPMI's `td_pigd`?

Answer. PPMI's curated `td_pigd` is coded two-class (1 = TD, 2 = non-TD), collapsing Indeterminate away, and the cut ships `pigd` with no matching tremor subscore — the ratio has a denominator and no numerator. The proposal requires all three classes and the continuous ratio.

Evidence in code. `docs/DATA_REQUIREMENTS.md`; the curated dictionary decode for `td_pigd`.

Follow-up: how do you know your recomputation is right? Two independent checks. Collapsed to binary it agrees with PPMI's own `td_pigd` on 4,431 of 4,431 visits. Against the archived three-class `td_pigd_old` it agrees on 1,595 of 1,597.

Q2. Why OFF-medication visits?

Answer. Levodopa suppresses tremor. Scoring an ON exam shifts the ratio toward PIGD and manufactures transitions that are pharmacological, not pathological. The Stebbins definition is specified on the OFF or untreated exam.

Evidence. `subtype.select_medication_state`; verified that all 9,954 removed rows are `PDSTATE == 'ON'`.

Follow-up: don't you lose untreated patients? No — the filter has an explicit `PDTRTMNT == 0` clause, unit-tested by `test_untreated_participants_are_kept`. At baseline 99.7% of the cohort is untreated and every one is retained.

Q3. Why actual dates rather than visit numbers?

Answer. `YEAR` is derived from `EVENT_ID` — a visit label, not elapsed time. In the real panel `V06` lands at 2.08 years and `V17` at 9.92. Using `YEAR` would force every interval to a whole number and destroy the interval-stratified analysis.

Evidence. `panel._add_time_axis`; PATNO 3010's trajectory.

Follow-up: how precise are the dates? Month resolution only — roughly ± two weeks, negligible at annual spacing. Stated as a limitation.

Q4. Why do unlabelled visits bridge instead of breaking the sequence?

Answer. Because a missed *assessment* is not a missed *patient*. PATNO 3010 has no label at `V08`; treating that as an endpoint would discard six years of subsequent follow-up. Instead `V06`→`V10` becomes one pair with a 1.92-year interval, and the interval is carried so the longer gap is visible to the analysis.

Evidence. `transitions.consecutive_pairs` line 1, and `test_unlabelled_visits_are_skipped_not_terminal`.

Follow-up: doesn't that assume nothing happened in between? It assumes nothing *observed* happened. That is precisely why intervals are stratified rather than pooled — a four-year pair is not treated as equivalent to a one-year pair.

Q5. Why not impute the missing subtype?

Answer. The implementation does not impute. The rationale is not explicitly documented in the repository. What *is* documented and tested is the sensitivity check: relaxing the rule to tolerate one missing item recovers only 44 visits and changes zero existing labels.

Interpretation (labelled as such). Imputing the subtype would mean imputing the outcome variable, which risks manufacturing the transitions the study is trying to measure.

Q6. Why are there fewer labelled visits than cohort visits?

Answer. 1,333 of 5,774 visits (23.1%) have no complete OFF-medication item set. Of these, 722 have no OFF exam recorded at all, 595 have an exam that could not be fully scored, and 16 lack Part II.

Follow-up: could this introduce selection bias? Yes, and its direction is known. Among the 1,253 unscorable OFF exams, `NP3PSTBL` — item 3.12, postural stability — is the missing item in 1,243 of them. The pull test is often unsafe in a patient with poor balance. Those exams come from more advanced patients: median Hoehn & Yahr 3.0 against 2.0 ($p = 1.4 \times 10^{-17}$). Postural stability is a *PIGD* item, so the visits lost are disproportionately those most likely to be PIGD. **The reported drift toward PIGD is therefore a lower bound.**

Q7. Why Wilson intervals?

Answer. Because one estimator has to serve both the pooled matrix and the interval-stratified table, and the normal (Wald) approximation breaks down in the second. In the pooled 3×3 matrix Wald would in fact be adequate — checked cell by cell, it never leaves $[0, 1]$ and agrees with Wilson to within 0.004. But the stratified table has row counts as low as 9, and there Wald returns a *negative* lower bound in five cells, for example Indeterminate→Indeterminate at 2.5–4.0 years ($k = 2$, $n = 16$): Wald [−0.037, 0.287] against Wilson [0.035, 0.360].

Even where Wald stays inside $[0, 1]$ it is wrong at small n, because it forces a symmetric interval around $\hat{p}$ when the true uncertainty is skewed. Switching estimator between tables would be worse than using the one that is valid for both.

Evidence. `transition_counts_with_ci`; `test_wilson_interval_stays_in_unit_range`.

Q8. Why Kaplan–Meier instead of a conversion percentage?

Answer. Follow-up ranges from one visit to fourteen years. A raw percentage cannot distinguish "did not convert" from "was not observed long enough". KM censors at the last labelled visit.

Follow-up: is censoring non-informative here? Probably not, and we say so. Dropout is heavy and, per Q6, scorability is tied to severity. The curves are truncated at year 12 where the risk set falls below 25.

Q9. Why a transition matrix at all?

Answer. It is the empirical object the Week 5–7 model must reproduce. Before fitting any model, we need to know what transitions look like: which states persist, which are transient, and whether movement is directional.

Follow-up: is the process Markov? Not established, and there is evidence against it. Iterating the pooled matrix gives a stationary distribution of TD 0.560 / Ind 0.103 / PIGD 0.338, whereas observed year-11 prevalence is 0.391 / 0.120 / 0.489. The gap is reported rather than hidden.

Q10. Does a TD → PIGD transition prove biological progression?

Answer. No, and the analysis is built to show why not. 35.2% of all observed changes reverse at the very next visit. A single observed transition is weak evidence about one patient.

Follow-up: then is any of it real? Yes. The movement is significantly asymmetric: TD→PIGD occurs 253 times against 141 in reverse, $p = 1.8 \times 10^{-8}$. Of 932 cross-state moves, 80.5% cancels out and 19.5% is net drift toward PIGD. The correct reading is a genuine drift observed through a noisy measurement — which is the case *for* a latent-state model, not against it.

Q11. Why patient-level rather than visit-level reasoning?

Answer. Transitions are only defined within a patient. `consecutive_pairs` does `shift(-1)` inside `groupby("PATNO")`, so one patient's last visit can never pair with the next patient's first. Patients with a single labelled visit (146) contribute nothing.

Follow-up: are patients weighted equally? No. A patient with 14 visits contributes 13 pairs; one with 2 contributes 1. The pooled matrix is therefore visit-weighted, not patient-weighted, and better-attended patients carry more influence. **This is not currently corrected for** and is a fair criticism.

13 What we have actually achieved

1. **A validated phenotype.** A three-class TD/PIGD label plus the continuous ratio, computed from raw MDS-UPDRS items for 4,441 visits, agreeing with PPMI's own derivations at 100% (2-class) and 99.87% (3-class).
2. **A longitudinal panel.** 1,080 patients, 5,774 visits, real elapsed-time axis, out to 14 years — `outputs/visit_panel.csv`.
3. **An empirical characterisation of transitions.** 3,361 pairs; matrices with Wilson intervals; interval stratification; reversion; time-to-conversion under two definitions; pathway analysis.
4. **Four findings that constrain the model.** Indeterminate is a crossing ($p_{\text{stay}} = 0.192$), PIGD is not absorbing (29.8% leave), 35.2% of changes reverse immediately, and transition probabilities scale with the inter-visit interval.
5. **A named, directional bias.** Label loss is concentrated in patients who cannot complete the pull test, making the reported drift a lower bound.
6. **Reproducibility.** Two commands regenerate every number; 74 unit tests; all parameters in one config file.

What has *not* been done. No model has been fitted. No parameter has been estimated. No prediction has been made. Everything above is descriptive characterisation of the observed data.

---

14 What comes next

Weeks 5–7 in the proposal: a baseline probabilistic model. The analysis in this document changes what that baseline should be.

Continuous-time Markov chain, not discrete
: Because TD→PIGD ranges from 0.100 to 0.289 depending only on the gap between visits, a chain indexed by "visit number" would encode PPMI's attendance schedule. Estimating a generator matrix Q and using $P(t) = e^{Qt}$ handles arbitrary intervals.

Decide the ratio question
: 16.3% of labelled visits have pigd_score = 0 and therefore no finite ratio. The recommended route is modelling the two subscores jointly and deriving the ratio's posterior, which keeps every visit.

Address the measurement noise explicitly
: 35.2% immediate reversion is the empirical justification for the proposal's observed = latent + noise structure.

Quantify the missingness bias
: Impute `NP3PSTBL`, or model the label as missing-not-at-random, to put a size on the lower bound in Q6.

Then the hierarchical Bayesian model

Population-level progression plus subject-level random effects, as in the proposal, followed by the σ_i-as-phenotype analysis in its second half.

Reference. Stebbins GT, Goetz CG, Burn DJ, Jankovic J, Khoo TK, Tilley BC. How to identify tremor dominant and postural instability/gait difficulty groups with the Movement Disorder Society Unified Parkinson's Disease Rating Scale. *Movement Disorders* 2013;28(5):668–70.